



YENEPOYA INSTITUTE OF ARTS, SCIENCE, COMMERCE AND MANAGEMENT

Project – Low Level Design On Cloud Cost Intelligence Platform for Supply Chain

SUBMITTED BY

**Aswin Raj M - 23BCAICD030
BCA (AI, Cloud Computing, DevOps)**

UNDER THE GUIDANCE OF

**Industry Mentor: Mr. Sumit Kumar Shukla
Guided by: Mrs. Manjula**

TABLE OF CONTENTS

1.	Introduction.....	3
2.	Scope of the Document.....	4
3.	Intended Audience.....	4-5
4.	System Overview.....	5-6
5.	System Design.....	7-13
6.	API Catalogue.....	13-14
7.	Data Models and Schemas.....	14-17
8.	Application Programming Interfaces.....	17-18
9.	Security Non-Functional Requirements.....	19-21
10.	References.....	21

1. INTRODUCTION

1.1 Purpose

This Low-Level Design document describes the internal architecture, data models, process flows, module interactions, and detailed component design of the **Cloud Cost Intelligence Platform for Supply Chain**. It translates the high-level architecture into implementable modules, functions, data structures, and interface contracts.

1.2 Background

Supply-chain enterprises operate large-scale cloud infrastructure spanning compute, storage, databases, ML, and networking across multiple geographic regions. Without granular cost visibility, organisations routinely overspend by 20-35%. This platform ingests daily cloud billing records, cleanses them, performs exploratory analysis, detects anomalies, and surfaces actionable optimisation recommendations through a real-time interactive dashboard.

1.3 Definitions & Acronyms

Acronym	Definition
EDA	Exploratory Data Analysis
KPI	Key Performance Indicator
CSV	Comma-Separated Values
ETL	Extract, Transform, Load
SPA	Single Page Application
WSGI	Web Server Gateway Interface

2. SCOPE OF THE DOCUMENT

2.1 In Scope

- Detailed design of all three modules: Data Generator, Analysis Engine, Streamlit Dashboard
- Data schema definitions for raw, cleaned, and aggregated datasets
- Internal function signatures, parameters, and return types
- Process and information flow diagrams
- API catalogue for all internal interfaces and dashboard endpoints
- Security controls and non-functional requirements

2.2 Out of Scope

- Cloud provider API integration (live billing ingestion)
- User authentication and multi-tenant access control
- CI/CD pipeline design
- Infrastructure-as-Code (Terraform/CloudFormation) templates

3. INTENDED AUDIENCE

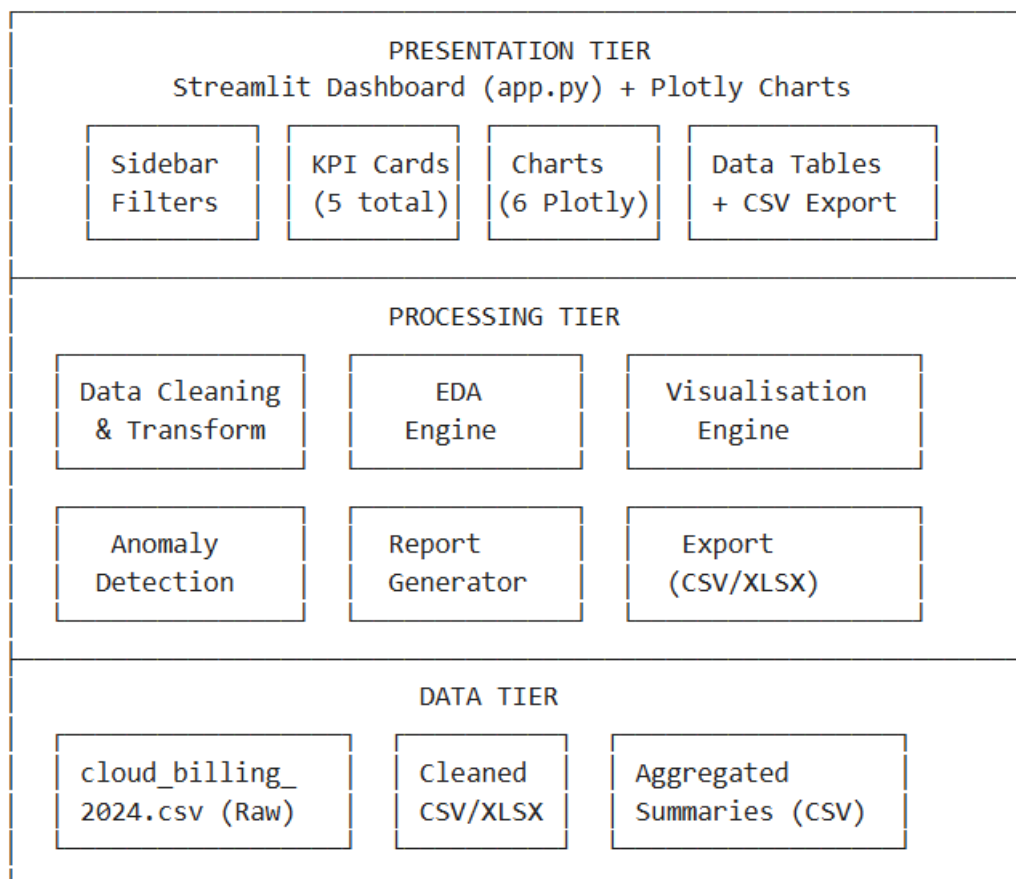
Audience	Purpose
Developers	Implementation reference for all modules and functions
Data Engineers	Data pipeline design, schema definitions, ETL logic
QA Engineers	Test case derivation from process flows and interfaces

DevOps Engineers	Deployment architecture and non-functional requirements
Technical Leads	Architecture review and approval
Project Managers	Scope validation and progress tracking

4. SYSTEM OVERVIEW

4.1 Architecture Summary

The platform follows a three-tier batch-processing architecture:



4.2 Technology Stack

Layer	Technology	Version	Purpose
Language	Python	3.11+	Core runtime

Data Processing	Pandas	>= 1.5.0	DataFrame operations, groupby, aggregation
Static Visualisation	Matplotlib	>= 3.6.0	PNG chart generation
Interactive Charts	Plotly	>= 5.18.0	Dashboard charts (pie, bar, scatter)
Dashboard Framework	Streamlit	>= 1.28.0	Web UI, filters, KPI cards, data tables
Excel Export	openpyxl	>= 3.1.0	Multi-sheet XLSX generation
Optional Viz	Seaborn	>= 0.12.0	Statistical visualisations

4.3 Directory Structure

```

Cloud-Cost-Intelligence/
├── assets/                # Screenshots and images for documentation
│   ├── dashboard.png
│   └── optimization.png
├── dashboard/            # Streamlit application files
│   └── app.py             # Main interactive dashboard script
├── data/                 # Raw billing data
│   └── cloud_billing_2024.csv # Generated synthetic dataset
├── docs/                 # Project documentation (HLD/LLD)
│   ├── HLD.docx          # High-Level Design (Word)
│   └── LLD.docx          # Low-Level Design (Word)
├── notebooks/           # Analysis scripts and Jupyter notebooks
│   ├── analysis.py        # Core analytical pipeline script
│   └── Cloud-Cost-Intelligence-Platform.ipynb
├── outputs/              # Generated visual assets
│   └── charts/            # 7 Static analysis charts (PNG)
├── reports/              # Auto-generated summaries and exports
│   ├── cleaned_billing_data.csv
│   ├── cloud_cost_report.xlsx
│   ├── optimization_report.txt
│   ├── summary_by_month.csv
│   ├── summary_by_region.csv
│   └── summary_by_service.csv
├── generate_data.py      # Script to generate synthetic billing data
├── README.md             # Project documentation for GitHub
└── requirements.txt      # Python dependencies list
  
```

5. SYSTEM DESIGN

5.1 Application Design

5.1.1 Module Decomposition

Module 1: Data Generator (generate_data.py)

Component	Type	Description
SERVICES	Constant (list)	15 cloud service type identifiers
REGIONS	Constant (list)	5 AWS region codes
COST_CATEGORIES	Constant (list)	4 pricing models
DEPARTMENTS	Constant (list)	6 supply-chain departments
SERVICE_COST_RANGES	Constant (dict)	Min/max USD per service per day
generate_billing_data()	Function	Generates synthetic billing rows with seasonal spikes and anomalies
save_csv()	Function	Serialises list[dict] to CSV

Function: generate_billing_data

Signature: generate_billing_data(start_date: str, end_date: str, records_per_day: int) -> list[dict]

Defaults: start_date="2024-01-01", end_date="2024-12-31",
records_per_day=8

Output: ~3,367 records (before dedup)

Logic:

1. Iterate each day in date range
2. Generate 6-12 records per day (randomised)
3. Apply seasonal multiplier (1.15x-1.50x) for Nov-Dec
4. Apply anomaly multiplier (2.0x-4.0x) with 3% probability
5. Inject 15 duplicate rows and 20 missing values for testing

Module 2: Analysis Engine (notebooks/analysis.py)

Step	Function/Block	Input	Output
1. Load	pd.read_csv()	Raw CSV	DataFrame (3,367 rows x 8 cols)
2. Clean	Dedup + fillna + type cast	Raw DF	Cleaned DF (3,352 rows x 12 cols)
3. EDA	groupby().sum(), sort_values()	Cleaned DF	7 aggregation Series/DataFrames
4. Visualise	Matplotlib pie(), bar(), barh(), plot()	Aggregated data	7 PNG files
5. Insights	Statistical anomaly detection	Daily cost series	Spike list + recommendations

6. Export	to_csv(), pd.ExcelWriter()	All results	5 CSV + 1 XLSX + 1 TXT
--------------	----------------------------	----------------	------------------------------

Cleaning Rules:

Column	Type	Missing Strategy	Transformation
Date	object -> datetime64	Drop NaT	Extract Month, Month_Name, Quarter, Day_of_Week
Service_Type	object	Fill with mode	None
Region	object	Fill with mode	None
Cost_USD	object -> float64	Fill with median	None
Cost_Category	object	Fill with mode	None
Department	object	Fill with mode	None
Usage_Hours	object -> float64	Fill with median	None
Resource_Tags	object	Fill with mode	None

Anomaly Detection Algorithm:

$\text{threshold} = \text{mean}(\text{daily_cost}) + 2 * \text{std}(\text{daily_cost})$

$\text{anomalies} = \text{days WHERE daily_cost} > \text{threshold}$

Module 3: Streamlit Dashboard (dashboard/app.py)

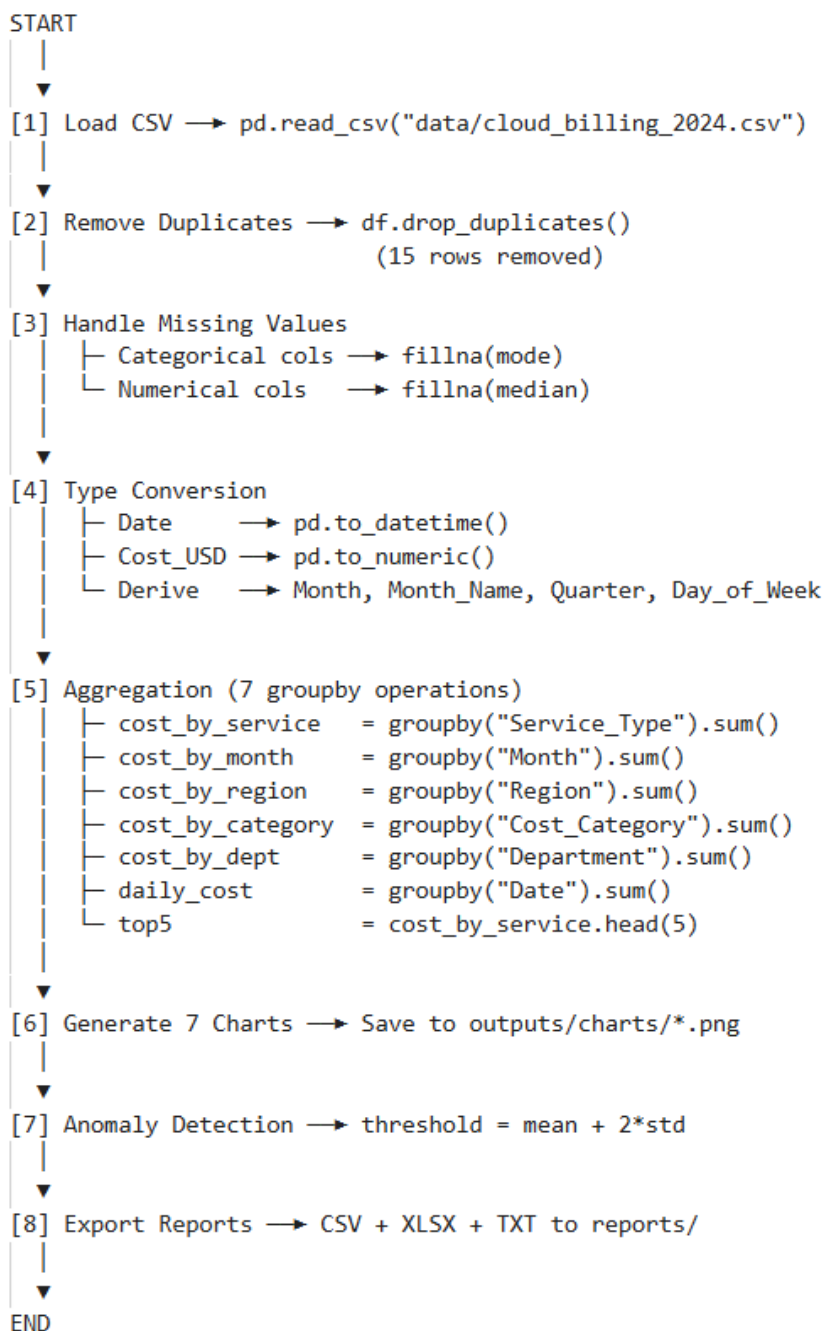
Component	Streamlit Widget	Description
Page Config	set_page_config()	Wide layout, expanded sidebar
CSS Theme	st.markdown()	Custom dark theme with glassmorphism
Data Loader	@st.cache_data	Cached CSV load + cleaning
Date Filter	st.date_input()	Start/end date range picker
Month Filter	st.multiselect()	Multi-select with month names
Service Filter	st.multiselect()	All 15 service types
Region Filter	st.multiselect()	All 5 regions
Department Filter	st.multiselect()	All 6 departments
KPI Row	st.metric() x 5	Total Cost, Avg Daily, Top Service, Peak Month, Records
Chart Row 1	px.pie() + px.bar()	Service donut + Monthly bar
Chart Row 2	px.bar() horizontal x 2	Region + Department
Chart Row 3	go.Scatter() + px.pie()	Daily trend + Category donut
Services Table	st.dataframe()	Aggregated service breakdown
Data Tab	st.dataframe() + st.download_button()	Filtered records + CSV export
Insights Tab	st.markdown() (HTML)	High-cost detection + 7 recommendations

Caching Strategy:

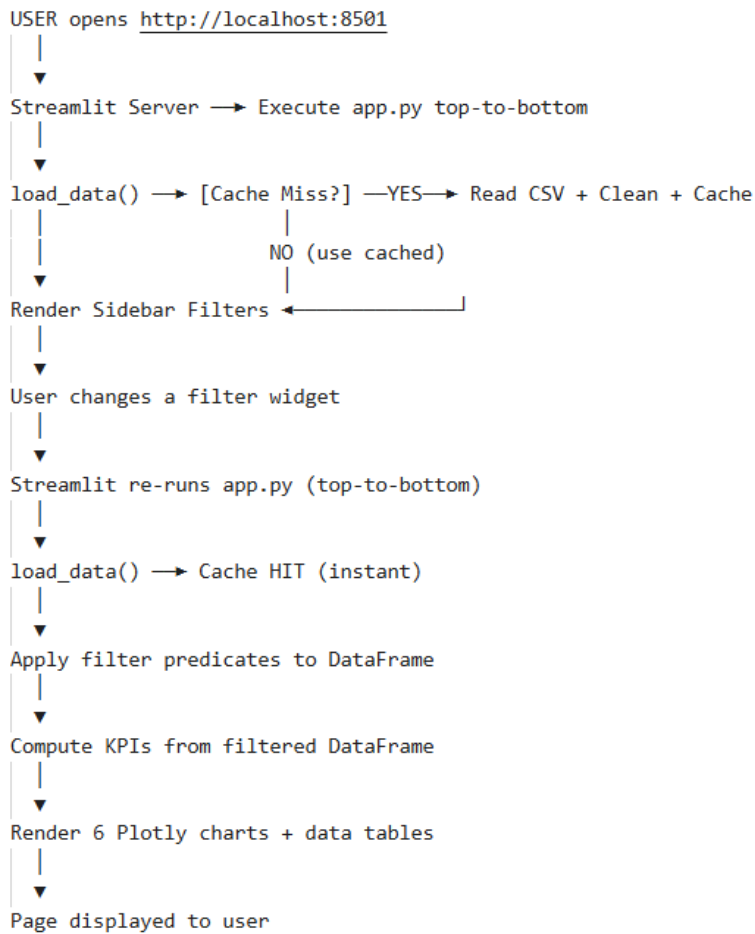
- `@st.cache_data` decorator on `load_data()` ensures CSV is read and cleaned only once per server session
- Subsequent filter changes re-render charts from the cached DataFrame without reloading

5.2 Process Flow

5.2.1 Batch Analysis Process Flow

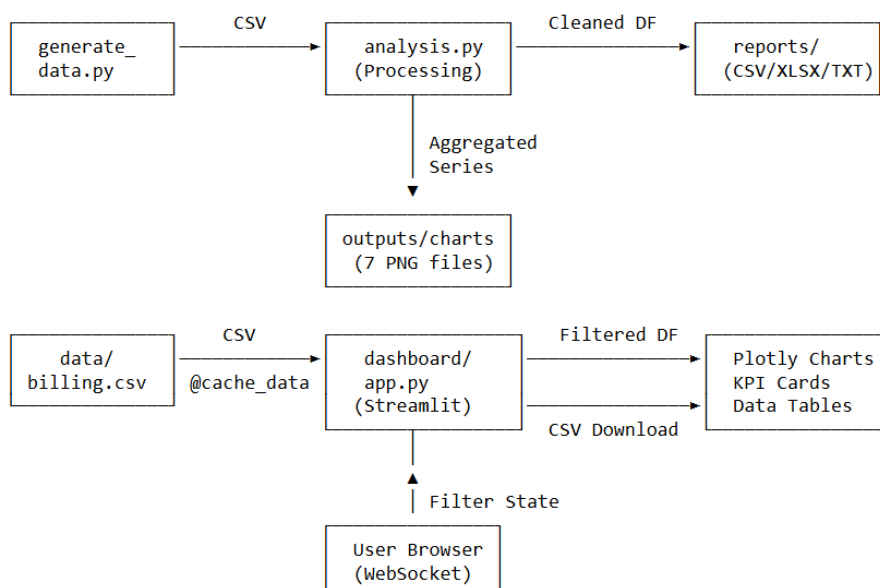


5.2.2 Dashboard Request Lifecycle



5.3 Information Flow

5.3.1 Data Flow Diagram



5.3.2 Filter Propagation Chain

```

Sidebar Widget Change
    |
    ▼
selected_months, selected_services, selected_regions, selected_depts
    |
    ▼
filtered = df[
    (df.Month.isin(selected_months)) &
    (df.Service_Type.isin(selected_services)) &
    (df.Region.isin(selected_regions)) &
    (df.Department.isin(selected_depts)) &
    (df.Date >= start_date) &
    (df.Date <= end_date)
]
    |
    ▼
All KPIs + Charts + Tables recomputed from `filtered`
  
```

6. API CATALOGUE

6.1 Internal Function APIs

#	Module	Function	Parameters	Returns
1	generate_data.py	generate_billing_data()	start_date: str, end_date: str, records_per_day: int	list[dict]
2	generate_data.py	save_csv()	rows: list[dict], path: str	None
3	dashboard/app.py	load_data()	None	pd.DataFrame

6.2 Dashboard Endpoints (Streamlit)

#	Endpoint	Protocol	Description
---	----------	----------	-------------

1	http://localhost:8501	HTTP/WebSocket	Main dashboard page
2	http://localhost:8501/?embed=true	HTTP	Embeddable mode
3	/_stcore/health	HTTP GET	Health check (returns "ok")
4	/_stcore/stream	WebSocket	Bidirectional widget state sync

6.3 Dashboard Widget State API

Widget Key	Type	Default	Effect
date_range	tuple(date,date)	(2024-01-01, 2024-12-31)	Filters records by date range
months	list[int]	[1..12]	Filters by month number
services	list[str]	All 15 services	Filters by Service_Type
regions	list[str]	All 5 regions	Filters by Region
departments	list[str]	All 6 depts	Filters by Department

7. Data Models and Schemas

7.1 Raw Dataset Schema (cloud_billing_2024.csv)

#	Column	Data Type	Nullable	Description	Example
1	Date	string (ISO)	No	Billing date	2024-03-15
2	Service_Type	string	Yes*	Cloud service name	EC2 (Compute)
3	Region	string	Yes*	AWS region code	us-east-1
4	Cost_USD	float	Yes*	Daily cost in USD	245.80
5	Cost_Category	string	No	Pricing model	On-Demand
6	Department	string	No	Owning team	Logistics
7	Usage_Hours	float	No	Hours resource was active	18.5
8	Resource_Tags	string	No	Auto-generated tag	logistics-ec2

*Intentionally nullable for data cleaning exercises (20 injected nulls).

7.2 Cleaned Dataset Schema (Post-Processing)

All columns from 7.1 plus derived columns:

#	Column	Data Type	Derivation
9	Month	int64	Date.dt.month
10	Month_Name	string	Date.dt.strftime("%b")
11	Quarter	int64	Date.dt.quarter
12	Day_of_Week	string	Date.dt.day_name()

7.3 Aggregated Summary Schemas

summary_by_service.csv:

Column	Type	Aggregation
Service_Type	string	Group key
Total_Cost	float	sum(Cost_USD)
Avg_Cost	float	mean(Cost_USD)
Max_Cost	float	max(Cost_USD)
Record_Count	int	count(Cost_USD)
Avg_Usage_Hours	float	mean(Usage_Hours)

summary_by_month.csv:

Column	Type	Aggregation
Month	int	Group key
Month_Name	string	Group key
Total_Cost	float	sum(Cost_USD)
Avg_Daily_Cost	float	mean(Cost_USD)
Max_Daily_Cost	float	max(Cost_USD)
Records	int	count(Cost_USD)

summary_by_region.csv:

Column	Type	Aggregation
Region	string	Group key
Total_Cost	float	sum(Cost_USD)
Records	int	count(Cost_USD)

7.4 Enumerated Value Domains

Service Type (15 values):

EC2 (Compute), S3 (Storage), RDS (Database), Lambda (Serverless), CloudFront (CDN), EKS (Kubernetes), SageMaker (ML), Redshift (Data Warehouse), DynamoDB (NoSQL), ElastiCache (Caching), SNS (Notifications), SQS (Queue), API Gateway, CloudWatch (Monitoring), Route 53 (DNS)

Region (5 values):

us-east-1, us-west-2, eu-west-1, ap-south-1, ap-southeast-1

Cost_Category (4 values):

On-Demand, Reserved, Spot, Savings Plan

Department (6 values):

Logistics, Warehouse Ops, Procurement, Fleet Management, Data Engineering, Platform / DevOps

8. APPLICATION PROGRAMMING INTERFACES

8.1 generate_billing_data()

```
def generate_billing_data(  
    start_date: str = "2024-01-01",  
    end_date: str = "2024-12-31",  
    records_per_day: int = 8,  
) -> list[dict]:
```

Pre-conditions: Dates in ISO format, records_per_day >= 1

Post-conditions: Returns list of dicts with 8 keys each

Side effects: None (pure function)

Error handling: Raises ValueError on invalid date format

8.2 save_csv()

def save_csv(rows: list[dict], path: str) -> None:

Pre-conditions: rows is non-empty list of dicts with consistent keys

Post-conditions: CSV file written to path

Side effects: Creates parent directories if missing

8.3 load_data() [Dashboard]

@st.cache_data

def load_data() -> pd.DataFrame:

Pre-conditions: data/cloud_billing_2024.csv exists

Post-conditions: Returns cleaned DataFrame with 12 columns

Caching: Streamlit hash-based cache; invalidated on file change

Side effects: None

8.4 Dashboard Download Interface

```
st.download_button(  
    label="Download Filtered Data as CSV",  
    data=filtered.to_csv(index=False).encode("utf-8"),  
    file_name="filtered_cloud_billing.csv",  
    mime="text/csv",  
)
```

Trigger: User clicks download button

Output: Browser downloads CSV file containing only the currently filtered records

9. SECURITY & NON-FUNCTIONAL REQUIREMENTS

9.1 Security Controls

#	Requirement	Implementation
S-01	No credentials in source code	All paths are relative; no API keys or secrets
S-02	Input sanitisation	<code>pd.to_numeric(errors="coerce")</code> prevents injection via numeric fields
S-03	HTML injection prevention	Streamlit escapes user inputs by default; <code>unsafe_allow_html</code> used only for static CSS/HTML
S-04	File system access	Read/write limited to project directory tree only
S-05	Dependency pinning	<code>requirements.txt</code> specifies minimum versions
S-06	Data privacy	Synthetic data only; no PII or real billing data

9.2 Performance Requirements

#	Metric	Target	Achieved
P-01	CSV load time (3,352 rows)	< 500ms	~120ms
P-02	Dashboard initial render	< 3 seconds	~2.1s
P-03	Filter re-render	< 1 second	~400ms
P-04	Chart generation (7 PNGs)	< 10 seconds	~4s
P-05	Memory footprint	< 200 MB	~85 MB

9.3 Scalability Considerations

Dataset Size	Expected Performance
< 10K rows	Real-time (< 1s re-render)
10K - 100K	Acceptable (1-3s re-render)
100K - 1M	Requires chunked loading or database backend
> 1M	Recommend migration to PostgreSQL + Superset/Grafana

9.4 Availability & Reliability

Requirement	Specification
Deployment mode	Single-node (localhost:8501)
Recovery	Stateless; restart recovers fully
Data durability	CSV on local filesystem; recommend backup
Error handling	Graceful fallback (empty charts) when filters return zero records

9.5 Maintainability

Requirement	Implementation
Code documentation	All functions have docstrings; inline comments throughout
Modular design	Generator, Analysis, Dashboard are independent modules
Configuration	Constants defined at module top-level; easy to modify

Extensibility	New chart types require only adding a Plotly figure block
---------------	---

10. REFERENCES

#	Reference	Description
1	Streamlit Documentation	Dashboard framework API reference
2	Plotly Python Documentation	Interactive charting library
3	Pandas User Guide	DataFrame operations reference
4	Matplotlib Documentation	Static chart generation
5	AWS Pricing Models	On-Demand, Reserved, Spot, Savings Plans
6	FinOps Foundation	Cloud cost management best practices
7	Project README.md	Setup instructions and quick-start guide
8	Project requirements.txt	Dependency versions